TrailSync — Trekking Management Application

Project Report

Student Details

Name:	<your-name>
Roll Number:	<roll_no>
Email:	<roll_no>@ds.study.iitm.ac.in
About:	Hi, I'm <your-name>, a Data Science student at IIT Madras. I have a strong interest in web development and love building full-stack projects, especially with Python and Flask.

Project Details

Project Understanding:

- Developed **TrailSync**, a web-based Trekking Management Application for adventure trek operations.
- Implemented **role-based access control** (Admin, Staff, User) with distinct portals for each role.
- Designed a complete workflow for:
  - Trek creation and lifecycle management by Admin
  - Staff assignment and participant / slot management
  - Trek browsing, booking, and cancellation by users
  - Booking history, trek history, and profile management
  - RESTful API endpoints for programmatic data access

AI/LLM Usage

Tools Used: Google Gemini

- Debugging errors and improving code correctness (<1%)
- Styling the UI using CSS and Bootstrap (8%)
- Implementing dashboard charts using Chart.js (3%)
- Understanding concepts and improving overall code structure

Technologies Used

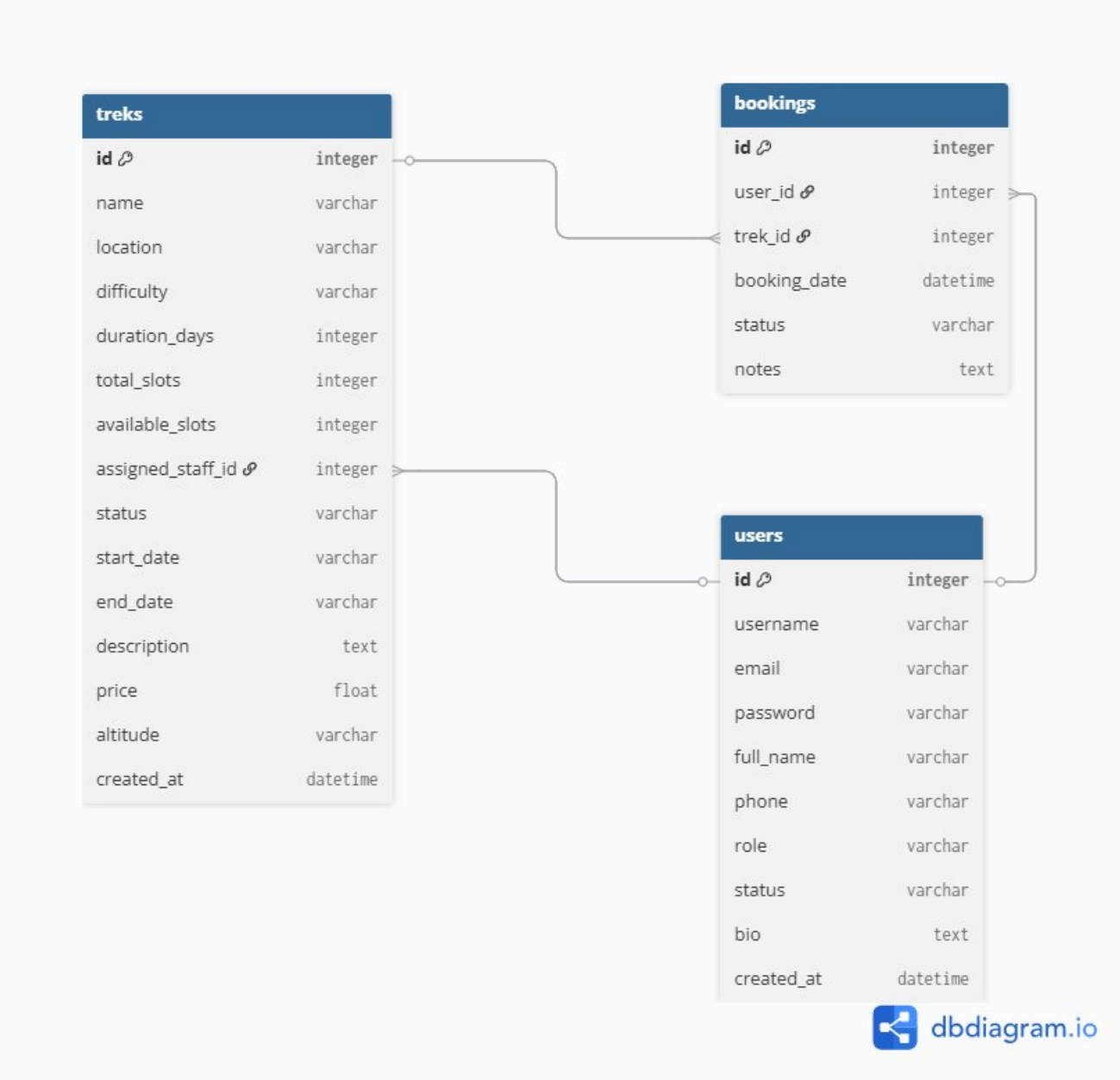
Backend	Flask, Flask-SQLAlchemy, Werkzeug
Frontend	HTML5, CSS3, Jinja2, Bootstrap 5, Chart.js
Database	SQLite
API	Flask Blueprint REST API (/api)

DB Schema Design

Tables and Structure:

- **USER**
  - *Relationships:*
  - • One-to-Many with Booking
  - • One-to-Many with Trek (as assigned staff)
- **TREK**
  - *Relationships:*
  - • Many-to-One with User (assigned staff)
  - • One-to-Many with Booking
- **BOOKING**
  - *Relationships:*
  - • Many-to-One with User
  - • Many-to-One with Trek

Schema Diagram:



Route Design

Authentication:

Method	Endpoint	Description
GET	`/login`	Open login / landing page
POST	`/login`	Authenticate user credentials
GET, POST	`/register`	Open & submit new trekker registration
GET	`/logout`	Terminate session

Admin:

Method	Endpoint	Description
GET	`/admin/`	Dashboard with statistics & Chart.js charts
GET	`/admin/treks`	View all treks (searchable)
GET, POST	`/admin/treks/add`	Create a new trek
GET, POST	`/admin/treks/<id>/edit`	Edit an existing trek
POST	`/admin/treks/<id>/delete`	Delete a trek
GET	`/admin/staff`	View all staff members
GET, POST	`/admin/staff/add`	Add a new staff member
POST	`/admin/staff/<id>/remove`	Remove a staff member
POST	`/admin/staff/<id>/toggle`	Blacklist / activate staff
GET	`/admin/users`	View all registered trekkers
POST	`/admin/users/<id>/toggle`	Blacklist / activate user
GET	`/admin/bookings`	View all bookings (searchable)

Staff:

Method	Endpoint	Description
GET	`/staff/`	Staff dashboard — assigned treks
GET, POST	`/staff/trek/<id>`	Manage trek: update status, view participants

User:

Method	Endpoint	Description
GET	`/user/dashboard`	User dashboard overview
GET	`/user/treks`	Browse treks (search & filter)
GET	`/user/treks/<id>`	View trek details
POST	`/user/treks/<id>/book`	Book a slot on a trek
POST	`/user/bookings/<id>/cancel`	Cancel an active booking
GET	`/user/bookings`	View active bookings
GET	`/user/history`	View completed trek history
GET, POST	`/user/profile`	View & update user profile

REST API:

Method	Endpoint	Description
GET	`/api/treks`	List all open / approved treks
GET	`/api/treks/<id>`	Get details of a specific trek
GET	`/api/users/<id>`	Get user details by ID

GET	`/api/bookings/<id>`	Get booking details by ID

Architecture and Features

Architecture Pattern:

app.py	Main Flask entry point; registers blueprints, creates admin user on first run
models.py	SQLAlchemy ORM models: User, Trek, Booking
routes/	Flask Blueprints — auth.py, admin.py, staff.py, user.py, api.py
templates/	Jinja2 HTML templates organised by role: admin/, staff/, user/, auth/
static/	CSS stylesheets and static assets (images)
seed.py	Demo data generator for development and evaluation — not for production

Features Implemented:

- **User Authentication & Authorization:**
  - Secure login and registration with session management
  - Password hashing using Werkzeug
  - Role-based access control: Admin, Staff, User
  - Custom login_required and role_required decorators per Blueprint
- **Admin Features:**
  - Dashboard with system-wide statistics and Chart.js visualisations (bar + doughnut)
  - Full trek lifecycle — create, edit, delete, assign staff, manage status
  - Staff management — add, remove, blacklist / activate
  - User management — view all trekkers, blacklist / activate
  - Global bookings overview with inline contextual search
- **Staff Features:**
  - Dedicated dashboard showing treks assigned to the logged-in staff member
  - Real-time slot tracking and participant roster management
  - Ability to update trek status (e.g., Open → Completed)
- **User (Trekker) Features:**
  - Browse and explore treks with search and filtering by location and difficulty
  - Single-click booking with real-time slot validation
  - Active bookings view with cancellation support
  - Permanent history of completed treks
  - Profile management — full name, phone, bio
- **REST API Endpoints:**
  - JSON responses for treks, users, and bookings
  - Enables programmatic / third-party access to core application data

Video

Video Link:
TrailSync_Project_Demo.mp4